

MILE STONE -3 : BACKEND AND FRONTEND

BACKEND MODULE

The backend module of **ExoHabitAI** is designed to deploy a trained machine-learning model as a usable web service. Instead of limiting predictions to a Jupyter notebook environment, the backend exposes the model through a Flask-based REST API. This allows external clients such as web browsers, frontend applications, or API testing tools to send exoplanet data and receive habitability predictions in a structured JSON format. The backend acts as a bridge between the trained model and real-world applications.

Backend Architecture and File Structure

The backend follows a modular design to improve readability, maintainability, and scalability.

app.py

- Acts as the main Flask application.
- Defines all API routes such as / and /predict.
- Loads the trained Random Forest model at startup.
- Handles incoming requests, performs predictions, and returns results as JSON responses.

utils.py

- Contains helper and utility functions.
- Responsible for:
 - Input validation
 - Feature preprocessing
 - Feature engineering required by the trained model
- Separating this logic keeps app.py clean and focused on request handling.

Model File

- The trained model is stored as a serialized file:
- random_forest.pkl
- The model was trained during the machine-learning phase and saved using joblib.

Model Loading and Integration

The Random Forest model is loaded once at backend startup using the joblib library.

```
model = joblib.load("random_forest.pkl")
```

Loading the model at startup improves performance by avoiding repeated disk access and ensures faster response times for prediction requests.

Input Features

The machine-learning model was trained using multiple planetary and stellar features describing physical, orbital, and stellar properties of exoplanets.

During prediction:

- All required features must be provided
- Feature names must exactly match those used during training
- Feature order must remain consistent

Maintaining strict feature consistency ensures reliable and accurate predictions.

API Endpoints

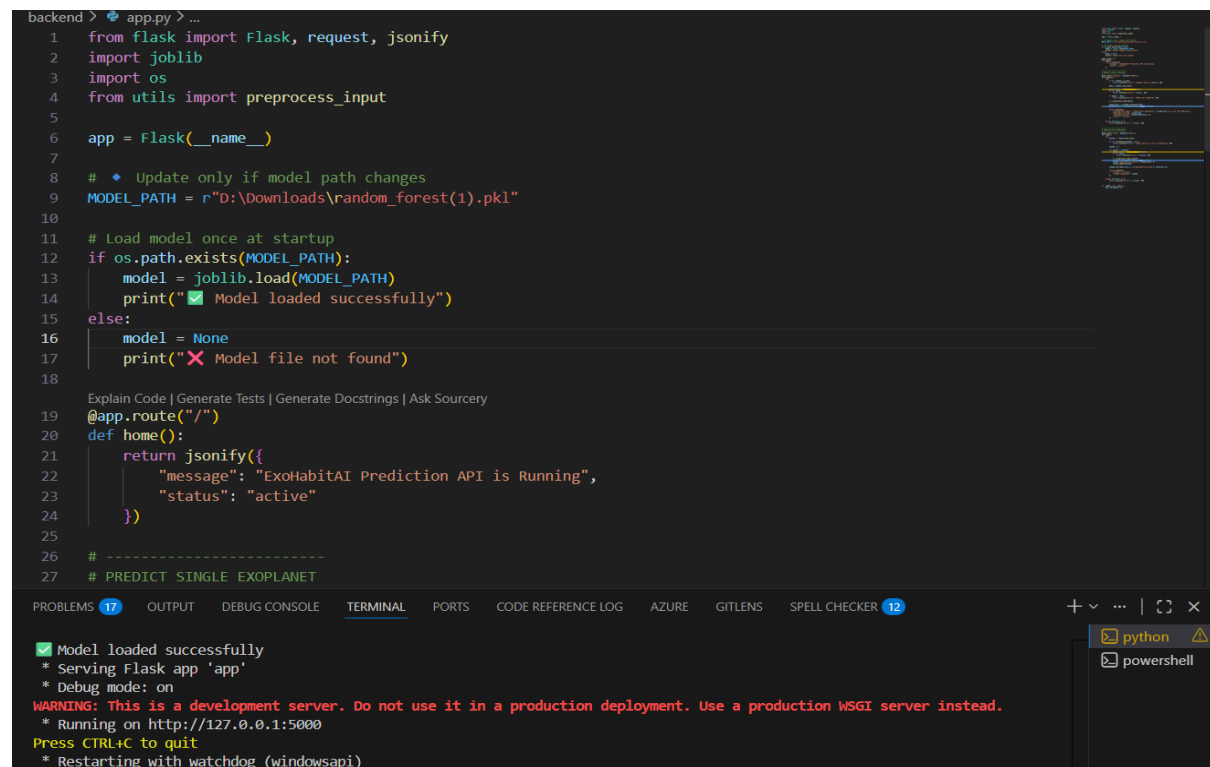
Home Endpoint (/)

- **Method:** GET
- **Purpose:** Verifies that the backend server is running

Example Response:

```
{
  "message": "ExoHabitAI Prediction API is Running",
  "status": "active"
}
```

This endpoint is mainly used for health checks and testing connectivity.



The screenshot displays a VS Code editor with a Python file named `app.py` and its terminal output. The code defines a Flask application with a `home` endpoint that returns a JSON response. The terminal shows the application running successfully on `http://127.0.0.1:5000`.

```
backend > app.py > ...
1  from flask import Flask, request, jsonify
2  import joblib
3  import os
4  from utils import preprocess_input
5
6  app = Flask(__name__)
7
8  # Update only if model path changes
9  MODEL_PATH = r"D:\Downloads\random_forest(1).pkl"
10
11 # Load model once at startup
12 if os.path.exists(MODEL_PATH):
13     model = joblib.load(MODEL_PATH)
14     print("✅ Model loaded successfully")
15 else:
16     model = None
17     print("❌ Model file not found")
18
19 Explain Code | Generate Tests | Generate Docstrings | Ask Sourcery
20 @app.route("/")
21 def home():
22     return jsonify({
23         "message": "ExoHabitAI Prediction API is Running",
24         "status": "active"
25     })
26
27 # PREDICT SINGLE EXOPLANET
```

Terminal Output:

```

✅ Model loaded successfully
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
```

Prediction Endpoint (/predict)

- **Method:** POST
- **Purpose:** Predict the habitability of an exoplanet

Workflow:

1. Accepts input data in JSON format
2. Validates the presence and correctness of required features
3. Preprocesses and converts the input into a model-ready feature vector
4. Passes the data to the trained Random Forest model
5. Returns the prediction result and confidence score

```
{  
  "status": "success",  
  "habitability_label": "Potentially Habitable",  
  "confidence_score": 0.59  
}
```

Input Validation and Error Handling

To ensure reliable predictions, the backend performs strict input validation before inference.

Validation checks include:

- Verification that all required fields are present
- Ensuring numerical values are correctly formatted

If invalid input is detected, the backend returns a clear and informative error message.

Backend Testing

The backend was tested using multiple methods:

- Web browser for testing the home endpoint
- PowerShell for sending POST requests to the prediction endpoint

Testing confirmed:

- Successful model loading
- Correct predictions for valid input
- Proper error handling for missing or invalid data

Conclusion

The ExoHabitAI backend successfully transforms a trained machine-learning model into a real-world application. By combining Flask, structured REST APIs, modular

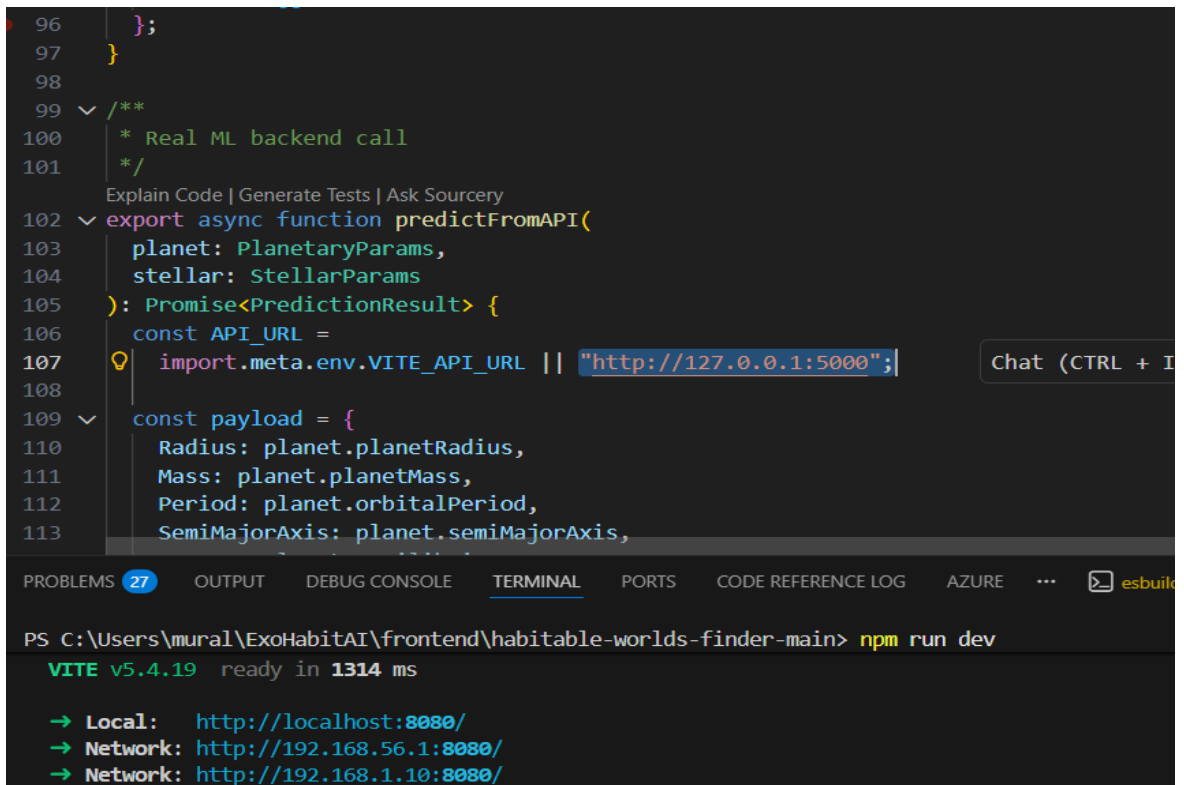
design, and strict input validation, the backend provides a reliable, scalable, and efficient system for predicting exoplanet habitability.

FRONTEND MODULE

Purpose of the Frontend

The frontend of ExoHabitAI serves as the user interaction and visualization layer. Its primary goals are:

- Collecting planetary and stellar input data
- Communicating with the backend prediction API
- Presenting AI results in an engaging and intuitive manner



```
96   };
97   }
98
99   /**
100    * Real ML backend call
101    */
102   export async function predictFromAPI(
103     planet: PlanetaryParams,
104     stellar: StellarParams
105   ): Promise<PredictionResult> {
106     const API_URL =
107       import.meta.env.VITE_API_URL || "http://127.0.0.1:5000";
108
109     const payload = {
110       Radius: planet.planetRadius,
111       Mass: planet.planetMass,
112       Period: planet.orbitalPeriod,
113       SemiMajorAxis: planet.semiMajorAxis,
```

PROBLEMS 27 OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG AZURE ... esbuild

PS C:\Users\mural\ExoHabitAI\frontend\habitable-worlds-finder-main> npm run dev

VITE v5.4.19 ready in 1314 ms

→ Local: http://localhost:8080/

→ Network: http://192.168.56.1:8080/

→ Network: http://192.168.1.10:8080/

Frontend Structure

• index.html

Root HTML file.

Defines the main mounting point (<div id="root">) where the React application loads.

• src/main.tsx

Application entry point.

Renders the main App component into the DOM.

- **src/App.tsx**

Main application wrapper.

Handles routing, layout structure, and global UI composition.

- **src/pages/Index.tsx**

Primary page of the application.

Integrates form inputs, result display, and backend prediction logic.

- **src/components/PlanetaryForm.tsx**

Handles planetary parameter inputs such as:

- Radius
- Mass
- Orbital Period
- Semi-major Axis
- Equilibrium Temperature

Manages user interaction and input validation.

- **src/components/StellarForm.tsx**

Handles stellar parameter inputs such as:

- Star Temperature
- Star Luminosity
- Star Type

Provides structured input fields for star-related data.

- **src/components/ResultsDisplay.tsx**

Displays:

- Habitability Score
- Classification Label
- Confidence Percentage
- Recommendations

- **src/components/PlanetRanking.tsx**

Displays ranking or comparison of planets based on habitability score.

Used for additional analysis and visualization.

- **src/lib/habitability.ts**

Core logic file.

Contains:

- `computeHabitabilityScore()` → Local heuristic model
- `predictFromAPI()` → Calls Flask backend ML API
- Utility functions like `clamp()` and `lerp()`

Acts as the computation bridge between frontend and backend.

- **`src/index.css`**

Controls:

- Tailwind CSS setup
- Global theme styles
- Fonts and typography
- UI customization

- **`package.json`**

- Project dependencies
- Build scripts
- Vite configuration
- Version control for libraries

- **`vite.config.ts`**

Vite configuration file.

Defines build settings, path aliases, and development server options

Frontend–Backend Communication

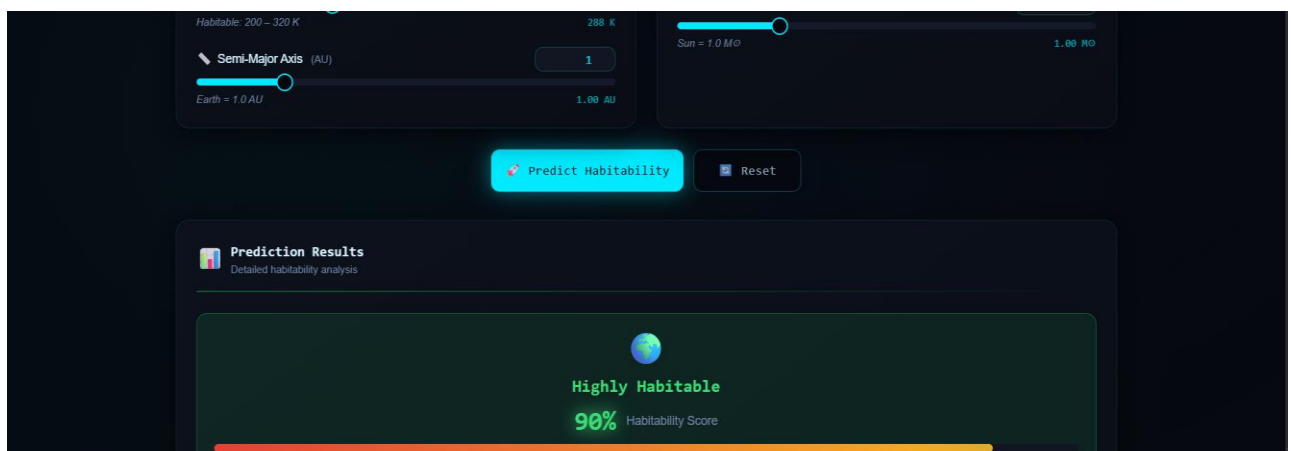
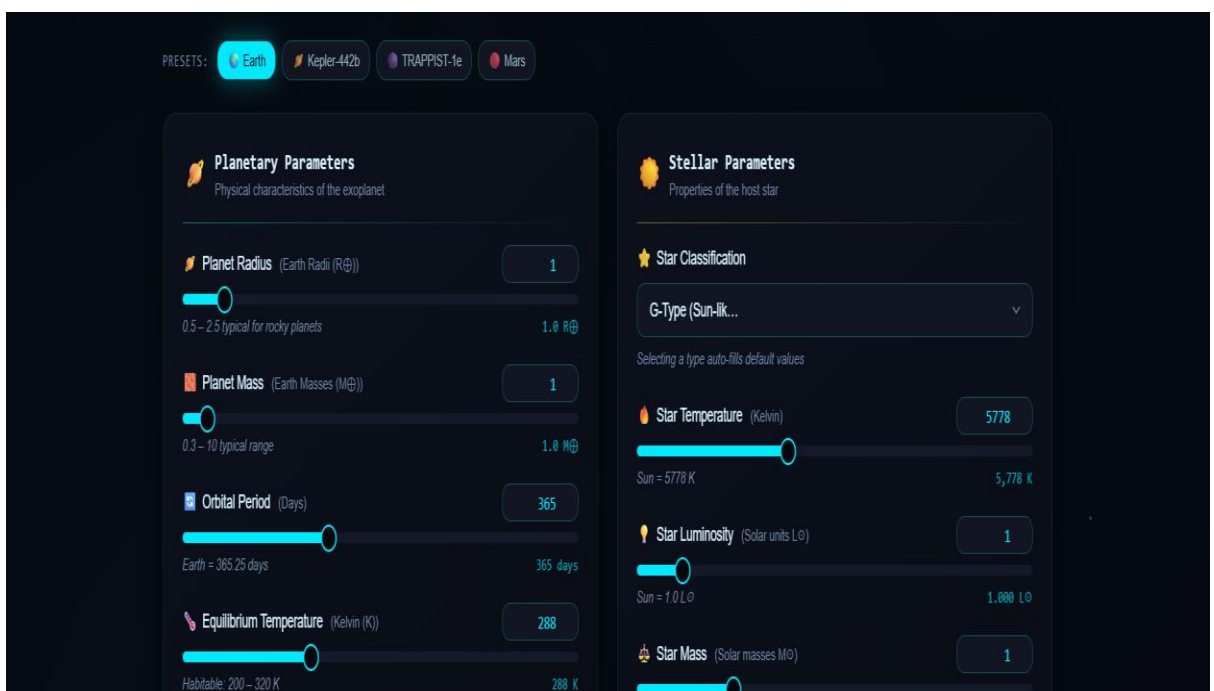
The frontend communicates with the backend using the Fetch API

- Sends user input as JSON
- Uses asynchronous requests
- Receives predictions without page reloads
- This allows a smooth and responsive user experience.

Result Visualization

After receiving the backend response:

- Habitability status is displayed clearly
- Confidence score is shown numerically
- UI components update dynamically.



Integration of Backend and Frontend

The strength of ExoHabitAI lies in its modular integration:

- Backend provides intelligence through machine learning
- Frontend delivers visualization and interaction

Both modules operate independently but communicate through well-defined APIs, ensuring:

- Clean separation of concerns
- Easier debugging
- Future scalability

Frontend Conclusion

The ExoHabitAI frontend successfully combines advanced visualization with real-time AI interaction. By integrating Three.js animations with a responsive prediction interface, it delivers an immersive and educational user experience while maintaining technical robustness.